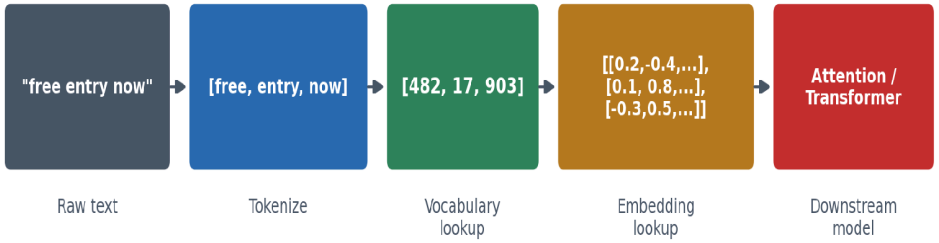
Tokens & Embeddings Crash Course

How raw text becomes something a neural network can learn from — and how this connects to attention and LLMs

1. The Core Problem: Text Isn't Numbers

Every model you've built so far takes numbers as input. Text is not numbers. Tokenization and embeddings are the two-step process that bridges this gap — and understanding both clearly is the foundation for everything else in NLP, up through attention and full Transformer-based LLMs.



The full pipeline: raw text is split into tokens, tokens are mapped to integer IDs via a vocabulary, and those IDs are used to look up dense embedding vectors — which is what any downstream model (attention, a classifier, etc.) actually operates on.

2. Tokenization

Tokenization splits raw text into discrete units. The choice of unit matters a lot:

- **Word-level —** Splits on words. Simple, intuitive — but the vocabulary can grow huge, and any word not seen during training is unrecognizable (handled with a fallback <UNK> token).
- **Character-level —** Splits into individual letters. Tiny vocabulary, never encounters an unknown symbol — but sequences become very long, and the model has to work much harder to reconstruct word-level meaning.
- **Subword / BPE —** A middle ground: common words stay whole, rare/unseen words get broken into smaller known pieces (e.g. "unhappiness" → "un" + "happi" + "ness"). This is what virtually every modern LLM actually uses — it keeps vocabulary size manageable while gracefully handling words never seen during training.

Your Part 1 notebook uses word-level tokenization — the right starting point for understanding the concept clearly. Your eventual GPT capstone will use subword tokenization, matching real-world practice.

3. Vocabulary & Special Tokens

A vocabulary maps every known token to a unique integer ID. Two special tokens matter beyond ordinary words:

- **<UNK> —** Stands in for any token not in the vocabulary. Without it, an unseen word at inference time would have nowhere to go.
- **<PAD> —** Fills shorter sequences up to a consistent length, since a batch of text needs uniform shape for a model to process it efficiently.

Critically — same leakage principle as every stage of this roadmap so far — a vocabulary must be built from TRAINING data only. Building it from the full dataset (including validation/test) would let information about unseen data leak into what the model "knows" exists.

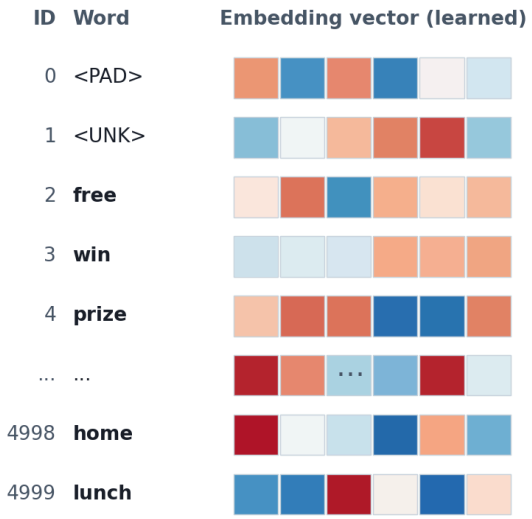
4. Embeddings

Why not just use the integer IDs directly?

A word's ID (e.g. "free" = 482) is arbitrary — there's no meaningful relationship between the numbers 482 and 483 just because they're numerically close. Feeding raw IDs into a model would imply a false ordering. Embeddings solve this by mapping each ID to a learned vector instead.

The embedding table

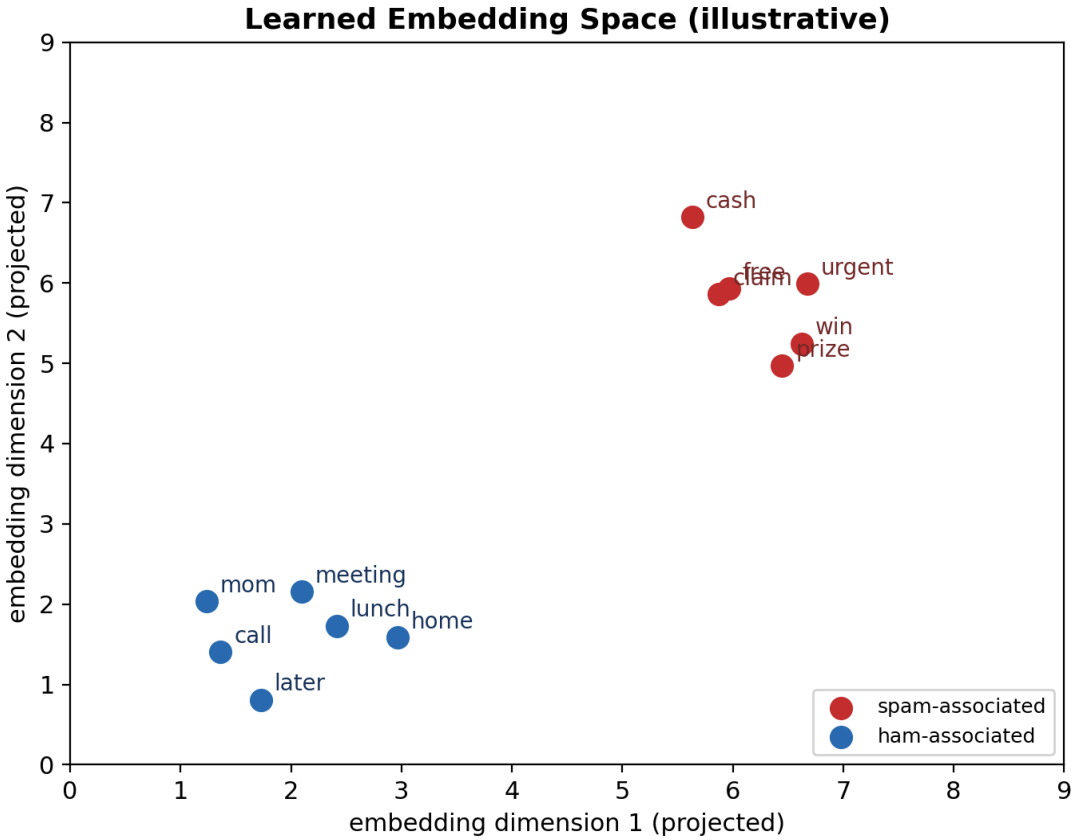
An embedding layer is really just a matrix — one row per vocabulary word, with as many columns as your chosen embedding dimension. Looking up a word's embedding is just reading its row.



Each vocabulary word maps to a row of learned numbers. Before training, these values are random and meaningless — training is what shapes them into useful representations.

Learned, not fixed — the key conceptual difference from TF-IDF

TF-IDF (Stage 6) assigns each word a single, FIXED weight based on frequency statistics — sparse, and with no notion that two words might be related in meaning. An embedding vector, by contrast, starts random and is LEARNED during training, exactly like any other weight in a network. Words that behave similarly in the training data — appearing in similar contexts — tend to end up with similar vectors, purely as a side effect of the model minimizing its loss. Nobody tells it "free" and "win" are related; it discovers this.



An illustrative projection of a learned embedding space. Words that tend to appear in similar contexts during training cluster together — this structure emerges from the data, it isn't hand-designed.

5. The Bigger Picture: Where This Leads

Tokens and embeddings aren't the end goal — they're the input representation every modern NLP system builds on top of, all the way up to large language models.

- **Mean pooling (what you just built) —** A simple way to combine a sequence of word vectors into one fixed-size representation — average them all together. Used in your Part 1 notebook. Real limitation: it discards word ORDER entirely ("dog bites man" and "man bites dog" average to the same vector).
- **Self-attention —** The mechanism that replaced mean pooling (and RNNs) in modern architectures. Instead of averaging, attention lets the model dynamically decide, for each token, which OTHER tokens in the sequence are most relevant — while preserving positional information via positional encoding. This is Part 2 of Stage 8.
- **The Transformer —** Stacks of attention + feedforward layers, repeated many times, operating on embedded token sequences. This is the architecture behind GPT, BERT, and effectively every modern LLM.
- **Large Language Models —** An LLM is, at its core, still doing exactly what your Part 1 notebook does at the input stage — tokenize, embed — just followed by dozens of stacked Transformer blocks instead of a small feedforward classifier, and trained on vastly more text.

The throughline worth holding onto: everything from here through your GPT capstone is built directly on top of the tokenize → embed pipeline in this document. The architecture downstream gets far more sophisticated, but the fundamental question — "how do I turn text into vectors a model can learn from" — is already answered by what you just built.

6. Quick Glossary

Term	Plain-English Meaning
Token	A discrete unit of text — a word, subword, or character, depending on the tokenization scheme.
Tokenization	Splitting raw text into tokens.
Vocabulary	The fixed set of all tokens a model knows, each mapped to a unique integer ID.
<UNK>	A special token standing in for any word not in the vocabulary.
<PAD>	A special token used to fill shorter sequences up to a consistent length within a batch.
Embedding	A dense vector of numbers representing a token, learned during training rather than fixed in advance.
Embedding table	The matrix of all embedding vectors — one row per vocabulary word — that a model looks up during the forward pass.
Embedding dimension	How many numbers represent each token (a hyperparameter — commonly 50-300 for small models, thousands for large LLMs).
Subword tokenization (BPE)	Splits text into pieces smaller than whole words (e.g. "unhappiness" → "un", "happi", "ness"), letting a model handle rare/unseen words gracefully. Used by virtually all modern LLMs.
Positional encoding	Extra information added to each token's embedding indicating its position in the sequence — needed because attention alone has no sense of word order.

7. Where This Connects to Your Roadmap

Your Part 1 notebook builds exactly the pipeline in Section 1's diagram, using your Stage 6 spam dataset so you can directly compare TF-IDF's fixed weights against learned embeddings on the same messages. Section 10 of that notebook has you inspect the actual learned vectors for spam- and ham-associated words — a real, hands-on version of the illustrative clustering shown in Section 4 above. The wrap-up's note on mean pooling's word-order limitation is deliberate — that gap is exactly what self-attention, in Part 2, is designed to close.